Corelang

Research Brief and Preliminary Outline

Prepared October 13, 2026 for Nathan Lewis II (OAL). Prepared by Claude.

How to read this

This brief runs the five-phase Corelang prompt in order: census, version history, rule anatomy, purpose and trajectory, then the Corelang outline. It ends with open questions for the next iteration.

Every claim carries one of these labels, so you can tell research from reasoning from design:

- **[SOURCED]** Taken from a source I found. Source list is at the end.
- **[DERIVED]** Worked out by me from sourced facts (counts, cadence arithmetic). Check the logic, not a citation.
- **[ESTIMATE]** A judgment call about a number or trend where sources disagree or are thin.
- **[SPECULATION]** A prediction or a guess about where something is heading.
- **[DESIGN PROPOSAL]** Part of the Corelang design itself. Yours to accept, change, or reject.
- **[DESIGN CONSTRAINT]** A boundary the design has to be built around.

Research was done on October 13, 2026. Several sources are blogs and news aggregators rather than official project pages. Where that matters, I say so.

The short answers

[ESTIMATE] How many programming languages are there? There is no single number. Counts range from about 250 to about 9,000, depending on what is counted. My working figure: roughly 9,000 languages documented across history, a few hundred to a couple of thousand in real use, and a couple of dozen dominating.

[DERIVED] Is there information on iterations and versions? Yes, and it is well documented for major languages. Python has 16 feature series from 3.0 through 3.15. Java has roughly 28 numbered feature releases since 1996. C++ has 7 published ISO editions. Rust has 4 editions. ECMAScript has 17 annual editions, with an 18th in progress.

What are the rules, what can you not violate, what can you do? Languages layer their rules: lexical, syntactic, semantic, runtime. The key finding for Corelang is that real languages already change their own rules, through a small set of repeatable mechanisms. Phase 3 and Phase 5 build on that.

Why are you asking? To build Corelang: a language whose rules are data, whose model of itself is kept up to date, and which can generate and test variants of itself.

Phase 1: Census

What counts as a language?

[SOURCED] The count depends entirely on the rules you set for what qualifies. Whether markup languages, dialects, aliases, and esoteric languages are included changes the answer by orders of magnitude.

Counts by source

- **[SOURCED]** TIOBE monitors about 250 popular languages.
- **[SOURCED]** Wikipedia's list of notable languages reached 700+ as of 2017. It excludes markup languages.
- **[SOURCED]** FOLDOC, an older computing dictionary, lists 1,000+, including aliases.
- **[SOURCED]** Rosetta Code reported 933 languages in February 2024. The site itself says it is aware of 943, and a July 2024 post cites 948.
- **[SOURCED]** PLDB, a public-domain database, holds 135,000+ facts about 4,000+ languages. Its own early snapshots were far smaller: 533 languages in 2017, then 3,800 to 4,266 entries later.
- **[SOURCED]** The Language List (HOPL) puts the all-time total at 8,945, the largest figure I found.
- **[SOURCED]** PLDB's author estimated 500 to 2,000 active general-purpose languages (2017 estimate).

One site claimed TIOBE shows 1,300+ languages. That conflicts with TIOBE's own roughly 250 tracked, so I excluded it.

My reading

- **[ESTIMATE]** About 9,000 languages documented across history. HOPL is the ceiling.
- **[ESTIMATE]** Most of that figure is unmaintained, historic, or one-off. The living set is a small fraction.
- **[DESIGN PROPOSAL]** The mix of dialects, markup, and aliases means no count is clean. Corelang should carry its own definition of "language" as a rule in its ledger, not inherit one.

Popularity snapshot

- **[SOURCED]** Python is No. 1 on TIOBE: 22.61% in January 2026 and 21.81% in February 2026.
- **[SOURCED]** C holds No. 2. Java is closing on C++ as of August 2026.
- **[SOURCED]** Rust entered the top 10 in July 2026 and held No. 10 in August.
- **[SOURCED]** MATLAB dropped out of the top 20 in August 2026 after a decade. TIOBE's CEO linked it to slower evolution than Python, C++, and Rust.

- [SOURCED] C# was TIOBE's Language of the Year for 2025.

Why it matters for Corelang: the unmaintained majority is a large training and testing corpus. It is also the group your prompt says Corelang should evolve forward.

Phase 2: Version history

Languages track their own change in several different ways. These are the patterns, with the major examples.

1. Annual feature series (Python)

- [SOURCED] Python ships one feature release per year in October, under PEP 602.
- [SOURCED] Python 3.14 came out October 7, 2025. Python 3.15.0 was released October 9, 2026, after a surprise third release candidate pushed it back from a planned October 1 date.
- [SOURCED] 3.15 headline changes: explicit lazy imports, new frozendict and sentinel built-in types, UTF-8 as the default encoding, a faster experimental JIT (about 7 to 8 percent on x86-64 Linux), and a new sampling profiler.
- [SOURCED] Python 3.16 (October 2027) is already set to get PEP 828, which lets async generators use yield from. It was accepted August 3, 2026.
- [SOURCED] Each release gets roughly 2 years of bugfix updates, then security-only fixes to about 5 years total.
- [DERIVED] 3.0 through 3.15 is 16 feature series, each with many patch releases (3.14 alone had reached 3.14.6 by June 2026).
- [ESTIMATE] The 3.15 release date comes from news items and an RC3 announcement, not the official page snapshot I could reach. Confirm at python.org.

2. Six-month releases with long-term support (Java)

- [SOURCED] Java 1.0 appeared January 23, 1996. A six-month cadence began with JDK 10 in March 2018.
- [SOURCED] Long-term support releases: 8, 11, 17, 21, 25. Java 25 shipped September 16, 2025. Java 26 shipped in March 2026. Java 27 shipped September 15, 2026.
- [DERIVED] Roughly 28 numbered feature releases from 1.0 to 27 (1.0 to 1.4, then 5 through 8, then 9 through 27). The early numbering is from my background knowledge.
- [ESTIMATE] Sources disagree on the next LTS. One says Java 27; the Wikipedia table lists Java 29 for September 2027. Two-year LTS spacing after Java 25 supports 29.

3. Formal standards committee (C++)

- [SOURCED] Published ISO editions: C++98, C++03, C++11, C++14, C++17, C++20, C++23. C++23 was published October 2024.

- [SOURCED] C++26 is listed as the next edition, with reflection, contracts, and std::execution as big features as of 2024. I could not confirm its final ISO publication status.
- [DERIVED] 7 published editions, with a roughly three-year rhythm since C++11.
- [SOURCED] The committee also publishes experimental technical specifications first. Several were later merged into the main standard (concepts, coroutines, modules, filesystem).

4. Annual spec snapshots plus staged proposals (ECMAScript)

- [SOURCED] ECMAScript 1 appeared in June 1997. ES2026 is the 17th edition. ES2027 is the 18th, in progress.
- [SOURCED] New features move through stages 0, 1, 2, 2.7, 3, and 4. Stage 4 means ready for the next edition.
- [SOURCED] Temporal reached Stage 4 in early 2026. Sources disagree on whether it lands in ES2026 or ES2027, though it already ships in Firefox 139 and Chrome 144. That is code running in the wild before the spec catches up.

5. Opt-in editions (Rust)

- [SOURCED] Editions exist: 2015, 2018, 2021, 2024, roughly every three years.
- [SOURCED] A project picks its edition in Cargo.toml. If unset, it defaults to 2015.
- [SOURCED] Crates on different editions can depend on each other. Editions can include incompatible changes such as a new keyword.
- [SOURCED] A migration tool, cargo fix, updates source code automatically.

6. Compatibility promise plus version-pinned semantics (Go)

- [SOURCED] Go 1 promises compatibility. Go 1.18 added generics, described by the team as the largest language change since Go 1.
- [SOURCED] If semantics ever had to change, the go line in go.mod would tell the compiler which meaning a module expects.

7. Contextual keywords (C#)

- [SOURCED] No new reserved keywords have been added since C# 1.0. New keywords arrive as contextual ones.
- [SOURCED] C# 2.0 added where, partial, global, yield, alias. C# 3.0 added the query words and var. C# 4.0 added dynamic. C# 5.0 added async and await.

Where this is recorded

- Python: python.org downloads and documentation-by-version pages.
- Java: Wikipedia's version history and Oracle roadmaps.
- C++: ISO and the standards committee pages.
- ECMAScript: Ecma International and the TC39 proposals repository.

- Rust: the Edition Guide. Go: the Go release notes.
- [SOURCED] Across languages: PLDB, as downloadable CSV, TSV, and JSON.

Patterns in how languages evolve

- [SOURCED] Breaking changes are rare and expensive. Python 2 to 3 (Python 3 shipped December 3, 2008) is the famous case, and the web is still full of Python 2 code.
- [DERIVED] Most change is additive, and the usual trick is to make new words contextual so old code keeps working.
- [DERIVED] Incompatible change is increasingly fenced behind an opt-in marker: Rust editions, Go's go.mod line.
- [DERIVED] Cadence has moved to fixed clocks: Python annual, Java every six months, ECMAScript annual.
- [DERIVED] Features often appear in implementations or technical specifications before they reach the standard.

Phase 3: Rule anatomy

A layered frame for rules

[DESIGN PROPOSAL] To compare languages, I split "rules" into layers. This is my own framing, meant as a starting schema.

- **Lexical:** what a word or symbol can be. Reserved words live here.
- **Syntactic:** what shapes are legal.
- **Static semantic:** types, scope, declared-but-unused checks.
- **Runtime:** what errors occur while running, and what is undefined.
- **Implementation and ecosystem:** limits, platforms, tooling, compatibility promises.

What is prohibited, and how it changed

- [SOURCED] **Python.** Keywords are always reserved. Python 3 turned print from a statement into a function, so the old form now raises a syntax error. The language also gained soft keywords: match, case, and underscore in 3.10, and type. A 3.15-era list shows lazy too. A soft keyword is only special in context, so match = True remains valid code.
- [SOURCED] **C#.** Reserved keywords are frozen since 1.0. New words are contextual, and an @ prefix lets you use a reserved word as a name. async and await were added in C# 5.0 without breaking code that used them as names.
- [SOURCED] **Rust.** New keywords arrive at edition boundaries. In 2018, async became a keyword, and old uses of the name were rewritten to the raw form r#async.

- [SOURCED] **Go.** A rule can tighten because a bug is fixed. Go 1.18 started correctly reporting some declared-but-not-used errors that earlier compilers missed, and began reporting an overflow for certain rune constants.
- [SOURCED] **Java.** Strong encapsulation of JDK internals (JEP 403) broke applications that reached into JDK classes through reflection.
- [SOURCED] **C++26.** Accepted changes include erroneous behavior for uninitialized reads, a reason string on deleted functions, and unnamed placeholder variables. Reading uninitialized memory moves out of the undefined category into a defined, flagged one.
- [SOURCED] **ECMAScript.** Rules change through the TC39 stage process, and implementations may ship ahead of the spec.

What each is permitted and meant to do

- [SOURCED] Python: broad reach, from scripting to data work. TIOBE notes specialized tools shave slices off that breadth.
- [SOURCED] C and C++: systems, embedded, and performance-critical software.
- [SOURCED] Java: long-lived enterprise platforms, with a predictable support model.
- [SOURCED] Rust: efficient, safety-focused systems code, but hard for non-experts to learn.
- [ESTIMATE] Go: simple, compatibility-first systems and services work.

Five ways a rule has been seen to change

[DERIVED] Reading across the examples, rule changes fall into five mechanisms. Corelang's mutability model in Phase 5 builds on these.

- **Reclassify the word:** reserved to contextual, or new words introduced as contextual (C#, Python soft keywords).
- **Fence it behind an opt-in:** editions (Rust), version-pinned semantics (Go).
- **Stage it:** proposals mature through stages before entering the spec (ECMAScript).
- **Deprecate, then remove:** the Python 3 transition, Java's encapsulation change.
- **Redefine the state:** undefined becomes erroneous (C++26); a bug fix makes a hidden rule enforced (Go 1.18).

Phase 4: Purpose and trajectory

What languages are evolving toward

[DERIVED] Six recurring directions show up across the sourced changes:

- [SOURCED] **Speed inside managed languages.** Python's experimental JIT and free-threaded builds (experimental in 3.13, officially supported in 3.14).
- [SOURCED] **Safety rails.** C++26 erroneous behavior, Java strong encapsulation, Rust's memory-safety focus.

- [SOURCED] **Concurrency.** Free threading in Python, std::execution in C++, yield from in async generators for Python 3.16.
- [SOURCED] **Ergonomics and expressiveness.** Pattern matching (Python 3.10), new type syntax, template strings (Python 3.14), iterator helpers (ECMAScript).
- [SOURCED] **Correctness of basics.** Temporal for dates and times, UTF-8 as Python's default encoding.
- [SOURCED] **Tooling folded into the language.** Python's new profiler, fuzzing in Go 1.18, a debugging library in C++26.

A method for predicting the next version

[DESIGN PROPOSAL] This method uses three inputs, each traceable to evidence: cadence, accepted proposals, and stated direction.

- **Cadence** gives when. **Accepted proposals** give what is nearly certain. **Stated direction** gives what is likely.
- Confidence falls as you move from the first to the third.

Predictions

- [SOURCED] Python 3.16 arrives October 2027 with PEP 828 included.
- [DERIVED] The next Rust edition lands around 2027 if the three-year rhythm holds (2015, 2018, 2021, 2024).
- [DERIVED] The next C++ standard follows around 2029 on the three-year rhythm, if C++26 completes on schedule.
- [DERIVED] Java 29, the next LTS, arrives September 2027 on the two-year LTS spacing.
- [SOURCED] ECMAScript 2027 includes Temporal-era and resource-management features, per one source.
- [SPECULATION] Python moves further toward free-threaded and JIT-by-default over the next few releases. This is a trend extrapolation, not a commitment from the project.
- [SPECULATION] More languages will adopt an edition or version-pin style opt-in, since it solves the breaking-change problem without splitting the ecosystem.

Phase 5: Corelang preliminary outline

[DESIGN PROPOSAL] Everything in this section is design. It draws on the evidence above but is yours to reshape.

5.1 Charter

[DESIGN PROPOSAL] Corelang's working model encompasses every language that came before it. Its syntax rules, restricted keywords, errors, and "cannot run" conditions are not fixed. They are modeled, tested, and allowed to change. Corelang builds a model of itself, generates variants, tests viability, and attacks its own design.

5.2 What "encompass" could mean

[DESIGN PROPOSAL] The phrase "every language" needs a working definition. Three levels are possible, and they can be adopted in order:

- **Represent:** every language's rules can be written as Corelang records.
- **Run or simulate:** programs from other languages can be modeled or executed through those records.
- **Generate:** new variants can be derived by changing the records.

[DESIGN PROPOSAL] Coverage can then be measured against the census: share of the roughly 9,000 documented languages represented, per level.

5.3 The Rule Ledger and the self-model

[DESIGN PROPOSAL] Each rule becomes one record in a ledger. A record carries:

- An identity, and the layer it belongs to (lexical, syntactic, semantic, runtime, ecosystem).
- Which language and version it comes from.
- Its status: reserved, contextual, permitted, forbidden, erroneous, or undefined.
- Its mutability class (see 5.4).
- Evidence: example programs that run and ones that fail.
- A change history and a confidence level.

[DESIGN PROPOSAL] Corelang then describes itself using the same record format. A "mirror" of Corelang's own rules sits in the same ledger, so the self-model and the language are one kind of object.

5.4 Rule mutability

[DESIGN PROPOSAL] Phase 3 found five real mechanisms. Corelang adopts them as its change operators:

- **Reclassify:** reserved to contextual to permitted.
- **Scope-fence:** a rule change applies to a file, module, or program, tagged with a version, so the old and new truth coexist.
- **Stage:** a proposed change passes through defined maturity steps before it becomes standing.
- **Deprecate, then retire.**
- **Redefine state:** undefined to erroneous to defined.

[DESIGN PROPOSAL] "This cannot run in this language, now it can" becomes a scoped, versioned ledger change, not an overwrite. Before a change is accepted, five checks run:

- **Compatibility:** does any program in the corpus change meaning?
- **Ambiguity:** does the grammar now parse two ways?
- **Reversibility:** can the old rule be restored or escaped?

- **Scope:** is the change contained to where it was intended?
- **Provenance:** is the reason and evidence recorded?

5.5 Corpus and ingestion

- **Living languages:** current rules, standards documents, and release notes, like the sources in this brief.
- **Unmaintained and historic languages:** catalog sources such as HOPL and PLDB.
- [DERIVED] **Same-task corpus:** Rosetta Code offers the same tasks solved in about 900+ languages, a natural test bed for cross-language checks.
- **Web and model artifacts:** HTML pages and language-model outputs as sources of language patterns, as in your prompt.
- [DESIGN PROPOSAL] Tiers: core living, standards-driven, unmaintained, esoteric. Each tier gets a different trust weight.

5.6 Variant generation and viability testing

[DESIGN PROPOSAL] The loop:

- **Mutate:** change one rule in a copy of the ledger (add, remove, or redefine a keyword; change a typing rule).
- **Regenerate:** rebuild the grammar and rules from the changed ledger.
- **Test:** run self-generated algorithms and Rosetta-style tasks across core languages and unmaintained ones.
- **Score:** parseability, task coverage, internal consistency, compatibility with the corpus, complexity cost, and whether the variant can still change again.
- **Select:** keep, quarantine, or discard.

[DESIGN PROPOSAL] To predict where a language goes next, the loop is seeded with the cadence, accepted proposals, and stated direction from Phase 4, and the surviving variants become ranked candidates with confidence levels.

5.7 Purpose and trajectory module

[DESIGN PROPOSAL] Each language gets a purpose record (what it was built to do) and a trajectory record (what it is evolving toward), using the six directions from Phase 4. This is how Corelang answers "where is your next version going".

5.8 Reframing primitives

[DESIGN PROPOSAL] Your example: a variable is a container, so what is an object that cannot be contained? Four candidate answers, each a different design direction:

- **Reference-only entity:** it can be addressed and acted on but never copied into a variable.
- **Flow object:** it exists only as a stream or event and has no resting state to hold.

- **Boundary object:** it defines the container's limits, so it sits outside every container by definition.
- **Relational identity:** its value is a relationship, and it has no stored content.

[DESIGN PROPOSAL] Reserved words: real languages already have the mechanism for a task-scoped reserved word, which is the contextual keyword. In Corelang, a word could be reserved for a task by a ledger grant that carries a scope and an expiry, rather than by permanent declaration.

5.9 Adversarial self-models

[DESIGN PROPOSAL] Corelang models adversarial versions of itself. Five starting adversaries:

- **Rule Poisoner:** proposes changes designed to create contradictions.
- **Ambiguity Maximizer:** builds programs with more than one valid parse.
- **Compatibility Breaker:** finds corpus programs whose meaning a change would alter.
- **Drift Auditor:** checks whether the self-model still matches actual behavior.
- **Escape-Hatch Abuser:** exploits escape syntax like @ or r# to smuggle in forbidden meaning.

[DESIGN PROPOSAL] Each adversary's findings feed back into the ledger as new checks.

5.10 Constraints to design around

- [DESIGN CONSTRAINT] "Every language" needs a boundary. The census shows no complete list exists, so Corelang should report coverage as a measured percentage against a named catalog.
- [DESIGN CONSTRAINT] Whether an arbitrary program runs, halts, or type-checks cannot be decided in general by any single model. Viability testing should therefore be empirical and sampled, with confidence levels, not a proof of viability.
- [DESIGN CONSTRAINT] A language modeling itself can only hold an approximation of itself. Self-models should be versioned snapshots, with the Drift Auditor measuring the gap.
- [DESIGN CONSTRAINT] Fully mutable rules risk losing any stable meaning. The evidence suggests mutability stays workable when each change is scoped, versioned, and reversible.

5.11 Suggested first steps

- Define the Rule Ledger record format.
- Seed it with five languages this brief already covers: Python, C#, Rust, Go, C++.
- Run one small variant experiment on keyword mutability, such as turning a reserved word into a contextual one and testing the result.
- Fix the viability scores and the first adversary.
- Decide the coverage definition from 5.2 before widening the corpus.

Open questions for the next iteration

- Is Corelang an executable language, a specification, or a modeling framework? The answer changes which of the three coverage levels in 5.2 is the target.
- Should this run as one prompt, or one prompt per phase?
- Should the adversarial models be a separate phase, or woven through each one?
- Does markup (HTML and similar) count as a language for the ledger? It is excluded from some counts and included in others.
- What does "evolve an unmaintained language forward" mean: toward its last standard, toward a modern peer, or toward a variant Corelang invents?
- Where does the first real test run, and what counts as passing?

Sources

Grouped by phase. Several are secondary or community sources; official pages are noted where I reached them.

Census

PLDB blog, "How many programming languages are there in the world?" https://pldb.io/blog/how-many-programming-languages-are-there-in-the-world.html

PLDB blog, growth of languages over time. https://pldb.io/blog/what-is-the-growth-in-programming-languages-over-time.html

PLDB repository readme. https://github.com/VictorEijkhout/pldb

Career Karma, "How many coding languages are there?" (cites HOPL 8,945). https://careerkarma.com/blog/how-many-coding-languages-are-there/

Rosetta Code. https://RosettaCode.org and https://en.wikipedia.org/wiki/Rosetta_Code

Factor blog, Rosetta Code post (948 languages). https://re.factorcode.org/2024/07/rosetta-code.html

TIOBE coverage by TechRepublic, Jan, Feb, Apr, Aug 2026. https://www.techrepublic.com/article/news-tiobe-august-2026-java-nears-c-plus-plus/

Versions

Python releases. https://www.python.org/downloads/ and https://www.python.org/doc/versions/

Python 3.15 coverage (release date, features). https://dev.to/tekmag/whats-new-in-python-315-3ea7 and https://zeli.app/ja/story/50021127

Python 3.15 RC3 announcement. https://www.warp2search.net/story/python-3150rc3-release-candidate-announced-stable-release-set-for-october-9-2026/

Python version guide. https://stevenpg.com/references/python-version-guide/

Java version history. https://en.wikipedia.org/wiki/Java_version_history

Java LTS timeline. https://hidekazu-konishi.com/entry/java_version_and_lts_timeline.html and https://thekodelab.com/en/posts/java-lts-evolution/

C++ standards. https://modernescpp.com/index.php/c26-the-next-c-standard and https://en.wikipedia.com/wiki/C%2B%2B26

C++ technical specifications. https://cppreference.net/cpp/experimental.html

ECMAScript editions and TC39. https://flaviocopes.com/ecmascript/ and https://mungomash.com/software/javascript/versions/

Rust Edition Guide. https://doc.rust-lang.org/beta/edition-guide/print.html

The Rust Book, Appendix E. https://rust.googlesource.com/book/+show/refs/heads/main/src/appendix-05-editions.md

Go 1.18 release notes. https://go.dev/doc/go1.18

Go generics expectations. https://groups.google.com/g/golang-dev/c/iuB22_G9Kbo

Rules

PEP 3105 summary. https://pynotes.bjdean.id.au/posts/pep-3105-print-function/

Python keywords and soft keywords. https://realpython.com/lessons/understanding-keywords/

Soft keywords discussion. https://discuss.python.org/t/changing-match-and-case-to-keywords/108127

Eric Lippert, reserved and contextual keywords. https://ericlippert.com/2009/05/11/reserved-and-contextual-keywords/

Microsoft, C# keywords. https://learn.microsoft.com/en-us/previous-versions/visualstudio/visual-studio-2012/x53a06bb(v=vs.110)